Due Date: 4/07/2026 Names: Aiden Wang

CSCI 240 PA_ Submission

Exercise 1
Pseudocode :

```
Algorithm IterativePreorder(T):
   If T is empty then return
   Initialize an empty Stack S
   S.push(T.root())

   while S is not empty do
       node = S.pop()
       visit(node) // print the node

       // Push children to stack in reverse order
       for each child in node.children() from right to left do
           S.push(child)
```

Algorithm Trace:

```
Step 1: S = [A]
Step 2: Pop A (Visit A). Push children D, C, B. S = [D, C, B]
Step 3: Pop B (Visit B). Push child E. S = [D, C, E]
Step 4: Pop E (Visit E). No children. S = [D, C]
Step 5: Pop C (Visit C). No children. S = [D]
Step 6: Pop D (Visit D). Push children G, F. S = [G, F]
Step 7: Pop F (Visit F). No children. S = [G]
Step 8: Pop G (Visit G). No children. S = []
Final Output: A B E C D F G
```

Exercise 2
Pseudocode:

```
Algorithm BreadthFirst(T):
if T is empty then return
Initialize an empty Queue Q
Q.enqueue(T.root())

while Q is not empty do
   node = Q.dequeue()  { p is the oldest entry in the queue }
   visit(node) //  print the node

   // Enqueue children in normal left-to-right order
   for each child in node.children() from left to right do
       Q.enqueue(child)  { add p's children to the end of the queue }
```

Due Date: 4/07/2026 Names: Aiden Wang

Algorithm Trace:

```
Initialization: Start with an empty Queue Q.
Enqueue the root node, A.
Q = [A]
Output = empty
Step 1: Dequeue A. Visit A. Enqueue its children from left to right (B,
C, D).
Q = [B, C, D]
Output = A
Step 2: Dequeue B. Visit B. Enqueue its child (E).
Q = [C, D, E]
Output = A B
Step 3: Dequeue C. Visit C. It has no children, so enqueue nothing.
Q = [D, E]
Output = A B C
Step 4: Dequeue D. Visit D.
Enqueue its children from left to right (F, G).
Q = [E, F, G]
Output = A B C D
Step 5: Dequeue E. Visit E. It has no children.
Q = [F, G]
Output = A B C D E
Step 6: Dequeue F. Visit F. It has no children.
Q = [G]
Output = A B C D E F
Step 7: Dequeue G. Visit G. It has no children.
Q = [] (Queue is now empty)
Output = A B C D E F G

Conclusion: The Queue is empty, meaning the algorithm successfully
terminates. The final visited order is A B C D E F G, which perfectly
matches the expected level-order output.
```

Due Date: 4/07/2026 Names: Aiden Wang

Exercise 3
Source code below:

```java
package PA5;

import net.datastructures.LinkedBinaryTree;
import net.datastructures.Position;

public class PA5_Ex3 {

   static class MyLinkedBinaryTree<E> extends LinkedBinaryTree<E> {
       @Override
       public Iterable<Position<E>> positions() {
           // Returns the pre-order iterable inherited from AbstractTree
           return preorder();
       }
   }

   public static void main(String[] args) {

       System.out.println("Modified by: Aiden Wang\n");

       /* ========================================
        * Test Case 1
        * ======================================== */
       MyLinkedBinaryTree<String> tree = new MyLinkedBinaryTree<>();

       // Constructing the binary tree exactly as requested:
       //        A
       //      /   \
       //     B     C
       //    / \   /
       //   D   E F

       Position<String> a = tree.addRoot("A");

       Position<String> b = tree.addLeft(a, "B");
       Position<String> c = tree.addRight(a, "C");

       Position<String> d = tree.addLeft(b, "D");
       Position<String> e = tree.addRight(b, "E");
       Position<String> f = tree.addLeft(c, "F");

       // Perform and print the Pre-order traversal
       System.out.println("Test Case 1:");
       System.out.print("Pre-order traversal for binary tree above: ");
       for (Position<String> p : tree.positions()) {
           System.out.print(p.getElement() + " ");
       }
       System.out.println("\n");

       /* ========================================
        * Test Case 2
        * ======================================== */
       tree.remove(c);
```

```
        // Perform and print the Pre-order traversal on the updated tree
        System.out.println("Test Case 2:");
        System.out.print("Pre-order traversal for updated binary tree
without node C: ");
        for (Position<String> p : tree.positions()) {
            System.out.print(p.getElement() + " ");
        }
        System.out.println();
    }
}
```

Input/output below:

```
/Users/guoqingwang/Library/Java/JavaVirtualMachines/temurin-22.0.2/Contents/Home/bi
Modified by: Aiden Wang

Test Case 1:
Pre-order traversal for binary tree above: A B D E C F

Test Case 2:
Pre-order traversal for updated binary tree without node C: A B D E F

Process finished with exit code 0
```

Answer for Question 1 :
Preorder: Root → Left Subtree → Right Subtree.
Postorder: Left Subtree → Right Subtree → Root.
Inorder (Binary Trees Only): Left Subtree → Root → Right Subtree.
Level-order (Breadth-first): Uses a Queue to visit nodes level by level, from top to bottom, left to right.

Preorder Traversal (Root, then Left-to-Right Subtrees): A, B, E, F, C, D, G, H, I
Post-order Traversal (Left-to-Right Subtrees, then Root): E, F, B, C, G, H, I, D, A
Level-order Traversal (Breadth-First, level-by-level): A, B, C, D, E, F, G, H, I

Answer for Question 2 :

Pre-order Traversal (Root → Left → Right): A, B, D, G, E, C, F
In-order Traversal (Left → Root → Right): G, D, B, E, A, C, F
Post-order Traversal (Left → Right → Root): G, D, E, B, F, C, A
Level-order Traversal (Breadth-First, level-by-level): A, B, C, D, E, F, G

Due Date: 4/07/2026 Names: Aiden Wang

Extra Credit – Option 1:
Pseudocode:

```
Algorithm IterativePostorder(T):
   if T is empty then return
   Initialize Stack S1 and Stack S2
   S1.push(T.root())

   while S1 is not empty do
       node = S1.pop()
       S2.push(node)

       // Push children to S1 in normal left-to-right order
       for each child in node.children() from left to right do
           S1.push(child)

   // S2 now contains the post-order sequence
   while S2 is not empty do
       node = S2.pop()
       visit(node)
```

Trace for the tree:

```
Step 1: S1 = [A], S2 = []
Step 2: Pop A to S2. Push B, C, D to S1. S1 = [B, C, D], S2 = [A]
Step 3: Pop D to S2. Push F, G to S1. S1 = [B, C, F, G], S2 = [A, D]
Step 4: Pop G to S2. S1 = [B, C, F], S2 = [A, D, G]
Step 5: Pop F to S2. S1 = [B, C], S2 = [A, D, G, F]
Step 6: Pop C to S2. S1 = [B], S2 = [A, D, G, F, C]
Step 7: Pop B to S2. Push E to S1. S1 = [E], S2 = [A, D, G, F, C, B]
Step 8: Pop E to S2. S1 = [], S2 = [A, D, G, F, C, B, E]
Step 9: Pop everything from S2 to print: E B C F G D A (Matches the
required output!)
```